

TECHNICAL UNIVERSITY OF MUNICH
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY
INFORMATICS

BACHELOR'S THESIS IN INFORMATICS

An Efficient Data Importer for Internet Measurements

Florian Wimbauer

TECHNICAL UNIVERSITY OF MUNICH
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY
INFORMATICS

Bachelor's Thesis in Informatics

**An Efficient Data Importer for Internet
Measurements**

**Ein effizienter Daten Importer für
Internetmessdaten**

Author:	Florian Wimbauer
Supervisor:	Prof. Dr.-Ing. Georg Carle
Advisor:	Christian Dietze, M. Sc. Tim Betzer, M. Sc.
Date:	February 23, 2026

I confirm that this Bachelor's Thesis is my own work and I have documented all sources and material used.

Garching, February 23, 2026

Location, Date

A handwritten signature in black ink, appearing to read 'S. Winklauer', written in a cursive style.

Signature

*The author additionally includes a
Qualified Electronic Signature pursuant
to EU Regulation 910/2014.*

ABSTRACT

Internet measurement data for the GINO research project originates from multiple providers. Because of that, researchers had to deal with various data formats and compression algorithms, before being able to process the data into a queryable form through importing it into a database. This heterogeneity has historically required researchers to rely on many small, proprietary and mostly manual scripts for processing and importing.

Through the implementation of a modular `python` framework that is able to process and import a variety of different files into a *ClickHouse* database, the import part of the research process is streamlined and automated. This leads to a reduction in required manual effort. Furthermore, the framework contains features like broken line detection, double import prevention and type validation, to improve the quality and consistency of the imported data. As the formats, layouts and compression algorithms of the raw data are subject to change, the framework is designed in a way that allows for easy expandability to support new formats and sources. Such expansions can be achieved by the user without needing to touch or understand the core functionality in detail, allowing the framework to become more capable in the future.

Through full parallelism capability and the possibility to import multiple files in one execution, the tool is optimized for high throughput and efficient CPU usage. The sophisticated logging mechanism allows for traceability and easy troubleshooting if problems occurred during import. The tool is also optimized for minimal memory and storage usage, leveraging in-database compression and iterator-functionality.

Experimental evaluation shows a speedup of the overall import process of around 5 to 10 times. The framework deals with common problems, leaving only the actual configuration to the user. This allows researchers to focus more on data-analysis instead of data-management.

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Research Questions	3
1.3	Outline	3
2	Background	5
3	Requirements	9
3.1	Speed	9
3.1.1	Easy Expandability for File Formats	10
3.1.2	Easy Expandability for Import Types	10
3.1.3	Import Speed	11
3.2	Resource Usage	11
3.3	Logging	12
3.4	Usability	13
4	Design	15
4.1	General Design Aspects	15
4.2	Pipeline Orchestration	16
4.3	Pipeline Stages	16
4.3.1	Decompressing Stage	17
4.3.2	Rowizing Stage	18
4.3.3	Analyzing Stage	19
4.3.4	Manipulator Stage	20
4.3.5	Import Stage	21
5	Implementation	23
5.1	Internal Data Management	26
5.1.1	DataContainer	26

5.1.2	Instruct	27
5.1.3	CH-Access	29
5.2	Modularity	29
5.3	Control Program	31
5.4	Logging and User Interface	32
6	Results	35
6.1	Deviation from the Design	37
6.2	Possible Improvements	38
7	Conclusion	41
7.1	Summary	41
7.2	Future Work	42
A	Appendix	45
A.1	List of acronyms	45
	Bibliography	47

LIST OF FIGURES

1.1	First lines of an exemplary import-file	1
1.2	Exemplary data-flow from download to database	2
2.1	Illustration of GINO downloads	5
4.1	Representation of the core pipeline of <i>Importheus</i>	15
4.2	Example for broken lines	20
5.1	UML Diagram of the Pipeline-Core	24
5.2	UML of the table-creation classes	25
5.3	Attributes of the internal data-object	26
5.4	Implementation of the instruction object	27
5.5	Syntax of the Instruction JSON file	28
5.6	Implementation of the clickhouse-access class	29
5.7	Excerpt of the file structure	30
5.8	UML Diagram of the Execution classes	31
5.9	Exemplary excerpt from the log-file	34
6.1	Excerpt of file with optional fields	38

LIST OF TABLES

2.1	Overview of used import-types	6
3.1	Overview of key requirements	9
4.1	Criteria for broken lines	19
5.1	Overview of subclass identifiers	30
5.2	Overview of log-levels	33
6.1	Overview of potential analyzer sub-stages	39

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

The modern Internet is an ever growing and changing global human endeavor. Getting a better understanding of its properties, structure and characteristics is essential, to keep it running, safe and reliable. For this purpose, researchers regularly conduct measurements, which are analyzed over time in a scientific manner, in order to trace the state of the Internet. This generates valuable insights that allows to adapt the Internet to arising challenges.

```
1 #####
2 # abuse.ch Feodo Tracker Botnet C2 IP Blocklist (Aggressive) #
3 # Last updated: 2025-12-30 13:56:31 UTC #
4 # #
5 # Terms Of Use: https://feodotracker.abuse.ch/blocklist/ #
6 # For questions please contact feodotracker [at] abuse.ch #
7 #####
8 #
9 "first_seen_utc","dst_ip","dst_port","c2_status","last_online","malware"
10 "2021-01-17_07:30:05","67.213.75.205","443","offline","2021-09-08","Dridex"
11 "2021-01-17_07:44:46","192.73.238.101","443","offline","2021-03-01","Dridex"
12 "2021-01-17_07:44:46","51.178.161.32","4643","offline","2023-01-30","Dridex"
13 ...
```

FIGURE 1.1: First lines of an exemplary import-file

As part of this scientific endeavor, the Global Internet Observatory (GINO) research project is conducted at the Chair of Network Architecture and Services at the Technical University of Munich. A key part of this project is the periodical accumulation of Internet Measurement data either through downloading from trusted third parties like other

research institutions from all over the world, or through conducting own measurements. As the data originates from many different sources and thus is very heterogeneous, direct usage is unpractical. Most of the data is compressed with various different data-formats underneath, like e.g. illustrated in Figure 1.1: a `.csv` file with comments and a header line. Another problem is the sheer size of the data. Most of the relevant sources provide multiple thousand data-lines in one file and often multiple files per day, making it impossible to recognize connections manually by just looking at it.

In order to gain any insight, a automated, database-based approach where the raw content of the files can be imported to compare data beyond the scope of a singular file would be practical. This can only be reasonably provided by some kind of processing like illustrated in Figure 1.2 that is either fully or semi-automatic in a way that no researcher has to touch individual lines or files. While a semi-automatic approach was used in the past (cf. Chapter 2) that could import similar files in one go, a fully automatic tool that imports all the desired data in one go without the need to pay attention to formats during import, is desirable and would improve the overall process of working with the data.

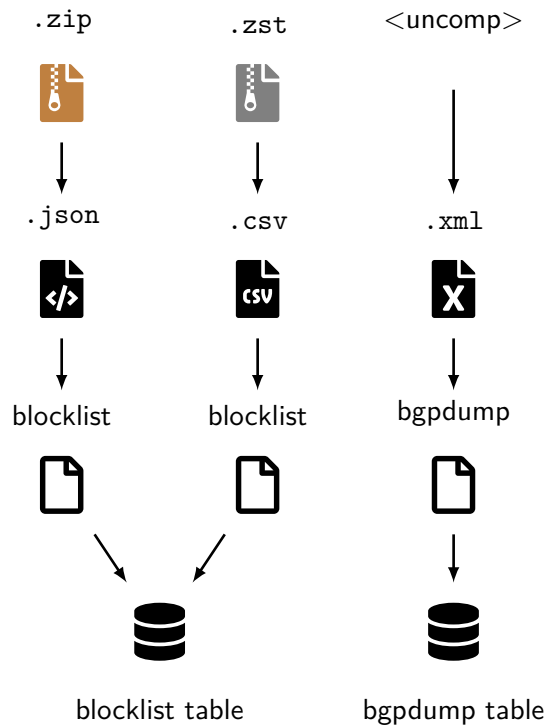


FIGURE 1.2: Exemplary data-flow from download to database

1.2 RESEARCH QUESTIONS

Since this thesis is fairly practical and focuses on implementation rather than analysis, the research-questions primarily represent questions that were asked during the design-process described in Chapter 4.

- **RQ-1:** How can the framework be designed in a way that allows for maximum flexibility for future use and modifications?
- **RQ-2:** How can faulty or incomplete data in imports be detected and handled?
- **RQ-3:** How much simpler is the new process that makes use of the framework in combination with *ClickHouse*?

1.3 OUTLINE

This project aims to provide import tooling as described in Section 1.1 by proposing and implementing a flexible and expandable framework that handles the import data from the time when it is downloaded, until it is processed into a general database where it can be analyzed.

First, the thesis describes the used technology and research problems in detail in Chapter 2. Then, it lists the requirements that result from analyzing the described problem set in Chapter 3. In Chapter 4 the core design elements of the framework are described together with its implementation in Chapter 5. In the end, Chapter 6 analyzes the results of the project, describing design concerns that arose during development. Lastly, Chapter 7 summarizes the whole thesis and gives an outlook on how the framework and the corresponding work-flow could be further improved in the future.

CHAPTER 2

BACKGROUND

The GINO research group was founded in 2016. Scientists of the group want to provide a better understanding of the current state of the Internet by analyzing data provided by public research communities and authorities as illustrated in Figure 2.1. Additionally, the group also conducts measurements and scans themselves. Their research focuses on understanding the structure, existing deployments and the security of the general Internet.

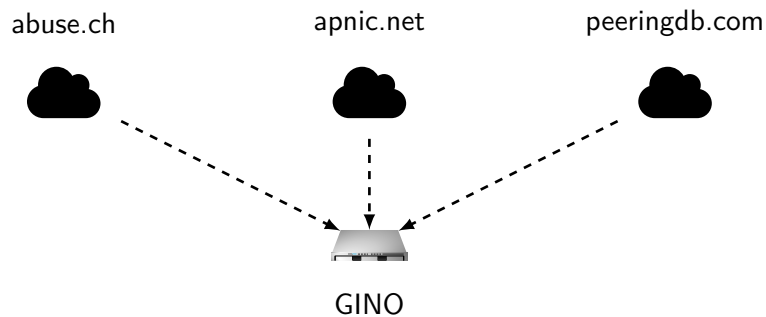


FIGURE 2.1: Illustration of GINO downloads

In order to be able to analyze the collected data, it is necessary for it to be converted in some kind of unambiguous structure like a database that allows for fast and easy querying. However, different data formats, compression formats and import-types make this step a non-trivial task that is only taken on for projects where it is obvious that the therefore created data-set is able to provide sufficient answers to the proposed research questions. In order to allow for a more exploratory approach on the analysis, especially across multiple import-types, reducing the overall complexity and expense of importing is key.

TABLE 2.1: Overview of used import-types

Name	Description	Source
<code>apnic-aspop</code>	autonomous system population data	Asia-Pacific Network Information Centre [1]
<code>blacklists</code>	data about hostile IPv6 addresses	abuse.ch [2]
<code>bgpdump</code>	BGP routing informations	GINO internal measurement
<code>peeringdb</code>	connection data of autonomous systems	peeringdb.com [3]

This work aims to provide this in form of a universal import tool that lays the foundation for automatic processing of all GINO-related measurement data regardless of their properties. It aims to simplify and speed up the process of researching with said data, allowing researchers to focus more on analysis instead of data-management. For the framework to meet these demands as a useful extension for the existing research and importing infrastructure, it needs to be able to handle the import for most of the data-types that are relevant for GINO without larger human intervention. Since the scope of this thesis is limited, the design focuses on the in Table 2.1 mentioned import-types as a proof-of-concept. Other scan data and derived data, e.g DNS-scans, ipv6-hitlists or QUIC-scans are currently not taken into account. However, the design allows for easy adaptation of more import-types at a later time as discussed in Section 3.1, fostering the goal of primarily creating a core-architecture for import handling that can be expanded and tailored to the needs of the researchers.

The Database Management System (DBMS) currently used by the GINO-researchers is *ClickHouse* [4]. Because of the analytical nature of the queries performed on the data, an Online Analytical Processing (OLAP) approach was chosen. The fact that *ClickHouse* is among the fastest analytical database on the market at the time of writing [5] in combination with its data-compression functionality and optimization for high ingestion rates makes it well-suited for this purpose. The latter two aspects are particular important, as the data can get very large very fast: often multiple thousand lines per file and multiple files per day.

Related Work

As of writing this thesis, there is not much related work that deals with a similar problem set. Before this work, the chair used several smaller, problem specific import scripts that were written by researchers themselves. Most of them were designed to only temporarily solve the task of importing one specific type (e.g. from Table 2.1) and were limited to only the one compression-algorithm and data-type that the researcher faced

at the time. This led to a scattered variety of multiple scripts with overlapping use-cases that often lacked critical features like double-import detection or line-validation which can lead to incorrect results in the worst case.

An example for one of those tools is the `python` script that S. Deusch [6] created for research on domain parking by analyzing `DNS-scans`. While importing the data he needed into *ClickHouse*, he also faced the same problems as mentioned in the beginning of this Chapter. As the importing itself was not the main goal of his work, the script's sole purpose was to transform the needed data into an analyzable form, explicitly tailored to his specific needs. However, his script was reused and loosely expanded for different projects. The implementation of this thesis is based on that work, among other existent and unciteable code snippets from the chair.

One of the few projects that implemented the core idea of this thesis at a larger scope is the Internet Yellow Pages project [7]. Their primary goal is to provide a database of Internet-measurement data to the scientific community of Internet researchers to be quarried for research purposes. In order to do so, they needed to tackle the tedious task of collecting many different import files and importing them into a single graph-database. They acknowledged that the processing of the different files from different sources is often an underestimated task that holds back exploratory research, especially if the advantage of completing the import is not obvious. They also choose to write the importer in `python`, however they use `Neo4j` [8] for their database since they lay special emphasis on graph-modeling.

CHAPTER 3

REQUIREMENTS

This chapter shines light on the detailed requirements summarized in Table 3.1 off of which we based the design of our import tooling.

TABLE 3.1: Overview of key requirements

No.	Name	Category
R1	New File Formats	Speed
R2	New Import Types	
R3	Import Speed	
R4	Memory	Resource Usage
R5	CPU	
R6	Database Size	
R7	Broken Lines	Logging
R8	Double Imports	
R9	Analysis	
R10	UI	Usability
R11	Auto-Import	
R12	Log-Files	

3.1 SPEED

Speeding up the whole process of importing the downloaded or measured data into a database that can be easily used for analysis by the scientists working on the GINO project is the primary goal of this project. Because of this, speed in terms of efficiency

plays a significant role in design decisions of the framework in many aspects that go beyond the scope of simple execution time, as described in the following paragraphs.

3.1.1 EASY EXPANDABILITY FOR FILE FORMATS

Since the data that needs to be imported is subject to unpredictable change, adaptability to new import-types is an important requirement. Researches are faced with different file formats like `.csv`, `.xml`, `.json` or `.yaml` that often are compressed in various different compression formats like `.zip` or `.zst`. It happened in the past, that the data- and compression-formats of a specific type of data from the same provider changed without notice or foreseeable cause from one day to another. Furthermore it is rather common that the data layout in a file changes. Such changes could surface in e.g. a different time encoding for date-columns (e.g. MEZ instead of UTC) or that the order of two columns in a `.csv` file gets interchanged or new ones get added.

Looking at this challenges, it is essential that the framework is well documented and written in comprehensible code. In addition to that, it needs to be easily expandable in the future. It needs to be possible for a researcher who doesn't know the framework yet and wants to use it, that he is able to implement support for a new compression format or a new file-type, if the framework is not supporting it yet (e.g. `.yaml` in addition to `.json`). Furthermore this should be achievable without the need for major modifications to the core of the framework, as this would not be scalable in the long term. This means it is necessary for the framework to auto-detect new expansions by itself. It has to know when to use which of the expansions autonomously in order not to jeopardize existing import workflows by later added functionality. This also requires the directives for a specific import-category not to specify specific data- or compression-formats explicitly.

3.1.2 EASY EXPANDABILITY FOR IMPORT TYPES

Since the import process and data content may vary significantly between different types of data that the framework should be able to work with, a way to specify the modalities of an import for each type is essential. For example a `blocklist` encoded in `.csv` and compressed as `.zip` would need different handling then an uncompressed `apnic-aspop` stored in `.json`. It may be noted that this isn't mostly because of the different compressions or file formats (as mentioned at the end of section 3.1.1 it would not be suitable to specify specific file formats for an import type for reasons of automation). It is primarily because of the completely different content and data structure of the two types. Therefore, a easy way of defining an import-type for each file is required which needs to be declared together with the path to the file that should be imported when calling the framework.

Adding compatibility to new import-types needs to be just as straight forward as adding

compatibility for new file formats. It again would not be useful if the core structure of the framework would require changes for new import types to be included. Therefore an approach where a new import-type can be defined in a standardized manner would be suitable, including auto-detection after implementation.

3.1.3 IMPORT SPEED

Import speed in this case describes how long it takes for one specific file to pass through every pipeline stage of the framework that is put together for its specific import-type. Although it is a requirement that the import process is not designed in an unreasonable slow or inefficient way, it is not of the highest relevance that the process is highly optimized. The framework is expected to support the import of multiple files in one execution. While there may be imports of only one file per execution in special occasions, most of the time multiple files will be given, each containing often more than several thousand lines of data. Because of this, it is expected that the import runs for a longer amount of time without constant supervision. In special cases where a whole new database is set up or after something went wrong, the tool might even run for multiple hours or days at a time. In all of those situations, it is not of primary interest if the process ran for 2h or 4h.

With the requirements from 3.1.1 in mind, it was decided to focus more on code reliability, readability and expandability instead of optimized single-file import speeds at this time.

3.2 RESOURCE USAGE

It is expected that the framework will regularly handle large amounts of data where most of it has the tendency to get even bigger after mandatory decompression for processing. This implies special consideration on how the framework handles limited resources like memory, CPU-time and storage.

The server at the chair that is reserved for the purpose of developing and testing this framework is called **hermes**. It is equipped with two Intel Xeon Silver 4114 server CPUs and provides 376GB of RAM (which is also used for storage in this case). Against this background, it would be highly anticipated that the framework is able to leverage the strong multi-threading capabilities of the CPU by optionally being able to import a given amount of files in parallel. Although the amount of RAM seems to be plenty, the sheer amount of data the framework needs to handle requires some sort of definable maximum of RAM that won't be exceeded at any time in execution, even if the decompressed memory usage of all files that should be imported in one execution succeeds this threshold. Those requirements allow for fast and reliable execution of the importer on

smaller machines, again making it more versatile for future use-cases.

Additionally, the size of the finished database itself should also be taken into consideration. Thus, a DBMS-service should be chosen which is optimized for analyzing large amounts of data for scientific purposes.

3.3 LOGGING

Keeping track on what the tool did is with no doubt one of the most important requirements that must be satisfied that is not directly related to the nature of the import-process itself. The tool is intended to be used in environments where it gets called once and runs on its own behalf for a longer amount of time. Thus, it needs to be designed in a way in which it is able to document the processing and importing of potentially millions of data lines in a human-comprehensible manner. This dynamic gets fostered even more by potentially automating the calling of the framework just like the download of the raw data is automated as mentioned in Section 3.4. In this case, the tool might run on its own for multiple days. Because of potentially arising inconsistencies in the database with this practice, it is essential to have at least an error log, better yet a complete log on what the program did to be able to detect potential errors that might have caused said inconsistencies. This is not only needed for finding the error, but also essential for reverting the database back to a consistent state.

Looking at consistency, it is particular important that the tool can detect, log and ultimately prevent double imports of the same file as this may severely tamper analysis results based on that data. Double imports might occur e.g. when a data source is not able to provide new data after the expected time period which would lead to a repeated download of the already known piece of data or when a researcher is not careful enough while crafting the instructions for a manual framework call. It is important to note that there may be exceptions where researchers might want to import the same data multiple times intentionally. This could be the case if e.g. no data is available for a specific day and the data from the last day should be used again to interpolate the missing day. For such cases, a option to forcefully import said file is needed.

It is expected that some of the data lines from the downloaded files are broken (e.g. a row in a `csv`-file is shorter than the header and the rest of the lines). For further analysis of this issues, it is also useful to collect meta-data of imported files, e.g. how many lines did a file contain and how many of them were broken. This allows to evaluate e.g. the error rate of a specific data-provider over time. As this information is more file specific rather than execution specific, it needs to be created for every single file that gets imported. This needs to happen individually for each file, completely independent

from the amount of files that get bundled together in on program-call, preferably in a database-like structure.

3.4 USABILITY

Closely related to the logging aspect are the requirements for overall usability. While the declared goal is to provide a better process and user experience compared to the current approaches for bringing raw downloaded data into analyzable format, there are a few specific aspects that the framework is required to fulfill in this domain.

Since the tool is primarily intended to work in automated Linux server environments, it is key for it to have a useful Command Line Interface (CLI) serving as the primary user interface. This provides a solid foundation for both manual execution by a researcher and automated execution by e.g. a shell script. Aside from different (optional) flags to control specific features of the tool, its main argument needs to be a proprietary data-structure that tells the framework which files to import with which directives. The data-structure must be designed in a way to support multiple files where each contains at least the path to the desired file as well as the import-type to tell the framework the modalities on how to handle said file.

Specifically for the situations where the framework imports multiple files, some kind of graphical progress-bar that displays the import progress on a file level would be of great value to the user. Preferably directly in the terminal. In addition to that, feedback over the command line regarding wrong formats or illegal options would improve usability. In Section 3.3, a few requirements for error-handling on a data-line level were already listed. Continuing this approach on a system level by gracefully aborting or secure handling of exceptions and avoidance of undefined behavior is key for a smooth user experience. This requires meaningful error-messages, e.g. when a file cannot be opened or is saved in a format that the framework currently doesn't support. The premise should be that problems with one file must not lead to a crash of the whole execution and thus to the non-import of other files in the import queue that would have worked under normal circumstances. In order to ensure this, extensive error handling, testing and debugging is required, to prevent sudden crashes. Logging those errors either directly to the command line during execution or optionally writing them into a dedicated `.log` file helps the user to retrace and solve occurring problems. Notably, this log should be separate from the data-line-error log mentioned in the previous Section as it serves a completely different purpose and doesn't need to be analyzable for scientific purposes. Its sole use lies in the provision of debug-data for the user to correct potentially faulty execution parameters and to retrace what the program did.

CHAPTER 4

DESIGN

This chapter describes the design decisions that were faced during initial development of the tool that will be called *Importhesus* in further Sections. After a careful analysis of the requirements stated in the previous Chapter, the tool was designed around the pipeline-process illustrate in Figure 4.1 which will be explained in detail in the following.

4.1 GENERAL DESIGN ASPECTS

In order to effectively limit memory usage, it would not be feasible to store complete files in memory during execution. This makes the straightforward design choice of executing a stage for a whole file and then handing the object to the next stages impractical. Because of this, the decision to implement a generator-based approach was made, allowing for easy memory control. Every Pipeline stage only gets a handle to the generator of the previous stage, that then can be altered in any way necessary before giving the handle to the next pipeline stage. Some stages also return a completely new generator. In the end, the generator gets consumed by the import-stage. With this approach, only one

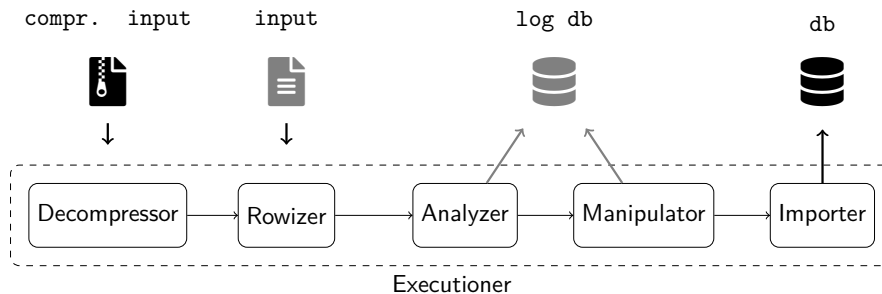


FIGURE 4.1: Representation of the core pipeline of *Importhesus*

row of data is in memory at every given time, making it very memory-efficient. The only exception are stages, where multiple reads from the previous generator are needed to craft a single element for the new generator (e.g. while skipping over comment-lines that don't need to be imported or merging data lines that are split over multiple text lines in the file, as it is the case with most JSON elements).

The overall goal of this design is to streamline the data iteratively on its way through the pipeline in a way that the underlying data- and compression-type from which the data initially stems from is not relevant for later pipeline stages. This is realized through the use of a uniform internal data-object that is used by all later stages.

4.2 PIPELINE ORCHESTRATION

One of the key features of the frameworks design is its easy expandability to new data-types, compression-types and import-types. In order to provide this, all pipeline-stages are designed in a way that implementation of functionality for new types is as easy as possible. This includes the execution unit (in Figure 4.1 described as "Executor") itself, whose task it is to iteratively call the individual stages of the pipeline and manage the exchange of data between them. Since not all of the import-types might need to use all of the stages, the executor can be customized for each type. This functionality allows for easy modification of the pipeline where e.g. the rowizer is executed two times or the decompressor is skipped completely for certain types. Especially the possibility to execute pipeline stages multiple times allows for a more complex import processes in the future.

In order to optimize throughput, CPU-usage and import speed, the whole pipeline can be parallelized on a individual file level. Although this doesn't improve the import speed of a singular file, importing multiple files in parallel gives a significant boost in runtime when importing a whole batch of files in one program-execution. The level of parallelism can be chosen for each program execution.

4.3 PIPELINE STAGES

Since the tool needs to support many different import-types where files can be encoded in many different ways as illustrated in Figure 1.2, the decision to choose a pipeline as the primary element of the framework was quite obvious. By taking a look at Figure 4.1, the pipeline stages are divided roughly into the vertical steps of the data flow described in Figure 1.2. This pipeline-approach is also in line with the requirement from Section 3.1.1 and 3.1.2 where easy expandability for new data formats and compression

was mentioned.

All stages except the import-stage are designed in a way to support a variety of formats. The decompression stage for example can decompress `.zip`, `.zst`, etc. Here, each of the implementations share the same interfaces, making them interchangeable. Furthermore, this approach allows for easy addition of new formats without the need to touch functionality of other formats.

The stages can be divided into two categories:

- **File Level:** The main goal of the file-level stages that get executed in the beginning, is to abstract the data-lines from different file-types and compressions into a standardized format in a way that they can then be used by the data-level stages. This allows the later stages to be designed independently from compression algorithms and file-formats. This is important, as it happens rather often that files with different file-types (i.e. different decompression- and rowizing-stages) need to be imported into the same database-table (i.e. same analyzing-, manipulating- and import-stages).

Because of this, the individual implementations of the stages are based on file- and compression-types and are independent from the import-type.

- **Data Level:** The key differentiating characteristic of the later executed data-level stages is that they only work with internal file objects, making them completely independent from potentially different sources of the data, only depending on the import-type. Their task is to validate, extend, correct and through that prepare the individual lines for the import into the database.

In the following, the design and functionality of the individual pipeline stages will be explained.

4.3.1 DECOMPRESSING STAGE

Most of the files that *Importheus* will be dealing with are compressed. In this case, the first important step for processing those files is to decompress them for the use in later pipeline stages. Consequently is this the stage, where a separate implementation for the most popular compression-algorithms like `.zip` or `.zst` is needed. It then auto-detects from the lexical ending of the file, which decompression algorithm should be used. That presupposes that all import-files are named according to Scheme 4.1.

$$\begin{array}{l}
 \langle \text{name} \rangle . \langle \text{type} \rangle . \langle \text{compression} \rangle \\
 \text{peeringdb_2021-01-16.json.zst} \\
 \text{Expected file naming scheme with example}
 \end{array}
 \tag{4.1}$$

Notably, the stage is not able to differentiate between a file that is compressed in a to it unfamiliar way, or not compressed at all. In both cases, it will propagate the file handle to the next pipeline stage for it to decide if it can be read directly or if the import of this file should be aborted due to readability issues.

Since it is the first stage in many cases, it is able to detect if a file exists at all. If it is obvious that the file is non-existent, the pipeline for this file will be stopped and skipped right away after logging the discovery to the UI (depending on the verbose level) and the logfile. As a file-level stage, it is independent from the import-type.

4.3.2 ROWIZING STAGE

The primary task of the rowizing-stage is to convert the data-lines into the internal data-object format in order to be universally readable for all later stages. Up to this point, the individual lines are still encoded in different file formats like `.json` or `.csv`. As shown in line 1-8 of the example in Figure 1.1, it is to be expected that some file-types like `.csv` contain commentary and empty lines that are not relevant for the import, while other files like `.json` have the information of one line scattered across multiple text lines. Because of this, the stage might consume multiple lines of the original file to produce one complete line of data that can be given to later stages. Additionally, the rowizer also fills in the header field of the internal data-object. In case the file does not provide any header information, the executor can provide a user-defined default header for this import-type.

Similar to the decompressor, the rowizer auto-detects which implementation to use from the second lexical file ending of the file, again depending on the naming-scheme illustrated in Formular 4.1. If the auto-detection fails e.g. due to violation of the naming-scheme, the executor for this import-type may again provide a default-type to determine with which implementation to proceed in this case.

This stage expects to read a raw file without any compressions present. Thus either the decompressor had to be run in advance (as intended by the default pipeline layout) or the user has to be sure that the given files are uncompressed already. If the rowizer gets a path to a file that he cannot read, he marks the file as invalid, logs it and stops the pipeline execution for this file. Aborting in this case is the only viable option, as all following stages are data-level stages that rely on data being provided exclusively in the internal data object format that initially only the rowizer is able to provide. This either happens if the file format is not known by the rowizer (i.e there is no implementation for it) or the decompressor propagated a still compressed file, as mentioned in the previous Section.

It is the second and last file-level stage and thus still generally independent from the import-type.

4.3.3 ANALYZING STAGE

This stage is by far the most sophisticated and compute-intensive one. It is the first stage where the general internal data object is ready and used. Before touching the file, the file-path is quarried in the import log, where a match would imply a potential double import, rendering the file invalid. Another case where this can happen is during the proceeding task, where the header field of the data-object is compared to the user defined header for this import-type. If they don't match, the file gets marked as invalid, the incident is logged and the pipeline execution for this file is stopped. This mechanism ensures that no entirely different file that got categorized wrong gets imported into a wrong database-table.

TABLE 4.1: Criteria for broken lines

Problem	Description	Line in Example 4.2
Not enough elements	A line has less elements than the header	2
Too much elements	A line has more elements than the header	3
Empty fields	The length of the line is header-conform but some fields are empty	5
Wrong type	The String can't be converted into the designated data-type	6

Once the conformity of the file is confirmed via the header-check, the stage proceeds to analyze every line of the data-object where it makes sure that every line conforms to the header on a syntax and data-type level. A line is declared broken if at least one of the problems mentioned in Table 4.1 occurs. See respective Examples in Figure 4.2. Notably, empty fields may be due to optional parameters in the data, thus not always indicating a broken line. Because of this, optional data-elements can be specified by the user during configuration of the analyzer import-type amongst the normal layout of the header. Of course, the analyzer gets every data-item of a line as **String** but it tries to internally interpret it as the user definition suggests. If this fails, the line is classified as broken. They then are removed from the generator that is given to the next stage and are put into a dedicated error-list of the data-object for further handling.

Since the header informations (field names and data-types) for the import-types are stored in the respective implementations of the analyzer, it is also its task to check, if

```

1 ...
2 "first_seen_utc","dst_ip","dst_port","c2_status","last_online","malware"
3 "2021-01-17T07:30:05","67.213.75.205","443","2021-09-08","Dridex"
4 "2021-01-17T07:44:46","192.73.238.101","443","offline","2021-03-01","Dridex","true"
5 "2021-01-17T07:44:46","51.178.161.32","","offline","2023-01-30","Dridex"
6 "2021-01-17T07:44:46","false","443","offline","2023-01-30","Dridex"
7 ...

```

FIGURE 4.2: Example for broken lines

the desired table in *ClickHouse* is already created and if the column names and types are coherent. If the table doesn't exist yet, the analyzer creates it. If it does exist but has different header fields (in respect to name or data-type), an error is logged and the pipeline execution of this file is stopped.

The last task of the analyzer is to make a log-table entry in *ClickHouse* reporting on the condition of the analyzed file (e.g. file-path, number of broken lines, total number of lines, import date) in accordance to the requirement of the second paragraph in Section 3.3. The logging is triggered after the returned generator is consumed completely by the import-stage.

As the handling of each individual line can be quite CPU-heavy it was discussed to parallelize this task on a line-level. However, this idea was discarded due to the management overhead and design challenges imposed by nested parallelism and the generator approach. Another reason against this was the requirement of Subsection 3.1.3, where the import speed itself was declared not to be a top priority.

4.3.4 MANIPULATOR STAGE

As mentioned as an example in Section 3.1.1, it happens quite often, that not only data types and files are subject to frequent and sudden change, but also the way the data is represented. Even if *ClickHouse* would probably be able to handle e.g. a differently formatted date-column, it could impose trouble to researchers when evaluating over such a dataset later on. Thus, the task of the manipulator is to again type-check all elements of all rows if they match with the defined data-type for the respective column in *ClickHouse*. If this is not the case, it first tries to repair the item (e.g. trying to convert a date from MEZ to UTC). If this is not successful, the line will be marked as broken and sorted out into the error list. Since it is able to sort out lines that the analyzer approved of, it is able to increase the amount of broken lines that get logged from the analyzing-stage after complete consumption by the import-stage.

Since the manipulator as a later pipeline-stage is normally scheduled after the analyzer, it relies on the internal data-object as input. Furthermore it depends on the analyzer

or the user that the data does no longer contain inherently broken lines (cf. Table 4.1). The stage is not designed to be a critical component. It doesn't house the ability to stop the execution of the pipeline as it is not able to detect any larger problems with the data that would endanger the import and thus making a controlled abort of the import of a file necessary.

4.3.5 IMPORT STAGE

The import stage is mandatory and is always the last stage data has to pass before getting imported into the database. It is completely independent from import- or file-types and thus not designed in modular way. It receives the final data-object where previous stages grant for it to be import ready and correct. It then consumes the generator in configurable batches, optimizing for import efficiency, and inserts them directly into *ClickHouse*. This stage has no influence on the data itself. Remaining lines in the error-list get discarded. If problems occur on import (e.g. database connection failed), they get logged into the log-file and the import of this file is gracefully aborted.

CHAPTER 5

IMPLEMENTATION

After taking a closer look on the general design aspects of the project, this Chapter describes the implementation of *Importheus* in detail, beginning from the user-interface all the way to internal data management.

The whole framework is implemented in `python`. Since most of the previous tools this project tries to replace were also written in this language, it allows later users to seamlessly integrate the new tool into their existing workflows. Used at the chair already, it fits in easily into the existing tooling eco-system for handling measurement datasets without the need to adapt to a new syntax. Furthermore this decision also made sense from a technical perspective: The framework is intended to run on the chairs GINO-server hardware primarily. Since that already supports `pip` and `python`, no additional setup except a few new libraries is necessary. In addition, `python` as a popular scripting language offers great capabilities for exactly such use-cases. Especially optimized libraries like `clickhouse-connect` and the option to work with user defined modules and packages that are automatically detectable by the system, make this language the obvious choice for this framework. Alternatively, languages like `C++` or `Rust` could have been used. The first would have made sense if speed would be a higher priority. As mentioned in Chapter 3, it is to be expected that the code gets inspected and extended by multiple people. In order to allow for easy understanding, the whole project is fully documented with `python docstrings`.

To ensure that each stage of the pipeline illustrated in Figure 4.1 can be executed via a universal `execute()` function, every pipeline stage derives from an interface-like super-class `base` as shown in Figure 5.1. This is particularly useful if one wants to add completely new stages in the future.

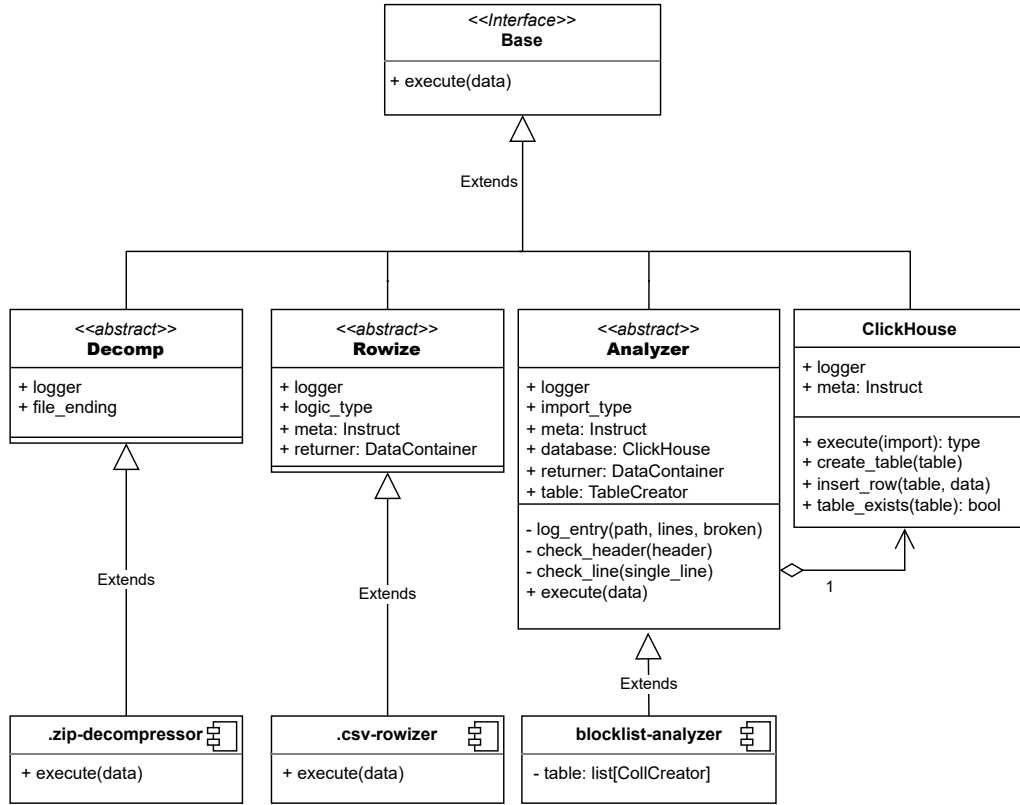


FIGURE 5.1: UML Diagram of the Pipeline-Core

Furthermore, the main-classes representing the pipeline-stages (mid-layer in Figure 5.1) are themselves abstract for reasons discussed in Section 5.2. Their specific functionality is implemented by the respective subclasses. Since the actual implementations will be adapted and created by the user, the whole system is designed in a way where as much of the general functionality of a stage as possible is housed in the abstract main-class. Especially the analyzing-stage relies heavily on that idea.

While the decoding and rowizing stages only implement their respective `execute()`-function in the subclass, the analyzing stage is far more complex but also more uniform. The only difference in the analyzing stage between import-types and thus between subclasses is the layout (i.e. fieldname and datatype) of the data. Apart from that, the analyzing-process is the same across import-types. Because of this, all of the analyzers functionality is directly in the main-class and doesn't need to be touched nor reimplemented in order to create new subclasses.

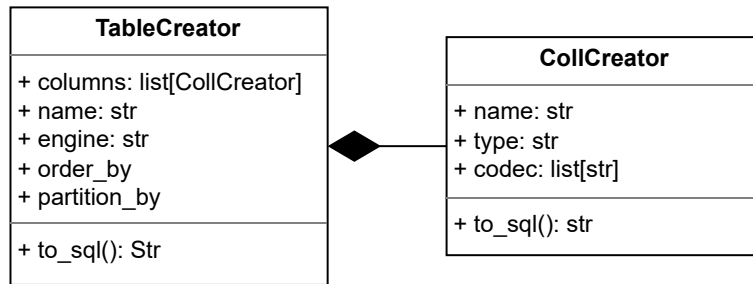


FIGURE 5.2: UML of the table-creation classes

Accordingly, the only element in the subclass itself is a list of **CollCreator** elements that fully define the layout of the database-table with, as seen in Figure 5.2, every entry representing a field of the table. The constructor of the main-analyzer-class then merges the list into a **TableCreator** element which contains functionality to generate a SQL-String able to create a table if sent to the DBMS. The **TableCreator** instance from the constructor is stored in the `table` variable of the analyzer class. Notably, the **CollCreator** houses the option to define the use of *ClickHouse Codecs* [9] that allows for memory optimization in the database once the data is imported.

Furthermore, the **CollCreator** list is also used during the integrity-check of each row. As described in Section 4.3.3, every item of every row gets checked for plausibility, meaning if the given **String** actually can represent the expected data-type for this field. However this cannot be done directly with the types given in the **CollCreator**, since the naming-syntax of *ClickHouse* data-types is different from the **python** syntax. To translate between them, the `typeDict` is used. This is a global variable in a separate `.py` file, that contains a `dict` that maps between a **String** representing a *ClickHouse*-data-type and a lambda-function that tests, if a **String** is or is not castable to the expected data-type using the `instanceof()` methodology. In summary, the `typeDict` maps between *ClickHouse* and **python** data-types while making a decision if the string-item can be of the expected type. Notably, the types of the items inside the rows of the `dataContainer` generators always stay of type **String**, since `clickhouse-connect` expects it this way. Apart from that, it is also easier and less error prone than a explicit cast into actual **python** data-types that *ClickHouse* inevitably has to cast again into its own types.

Multiple additional libraries like `ipaddress` and `datetime` were used to support all necessary data-types in **python**. It is to be expected that the control instance is not able to detect all problematic items in all lines, based on the approach's described nature.

Again illustrated in Figure 5.1, we see that the analyzer main-class contains a reference to `ClickHouse`, which is the same object that will get executed later on in the pipeline. This is necessary, as this object contains all the utility-function wrappers to communicate with the database. The analyzer needs those to make the log entries into the logging table, to create tables if necessary and to quarry the database. The latter is done to compare the headers of the desired table with the ones from the import-data and to check past log-table entries to prevent double imports.

5.1 INTERNAL DATA MANAGEMENT

Throughout the whole framework, meta-data and import-data needs to pass uniformly between multiple objects. This is implemented through multiple data-classes that are described in this Section.

5.1.1 DATACONTAINER

Arguably the most important aspect of this Section is the orchestration of import-data flow itself. For this, the data-class `dataContainer`, illustrated in Figure 5.3, is used (often referred to as *internal data object* throughout the thesis). As already repeatedly discussed, the tool uses a generator approach for the import-data, where the handle is stored in the `rawData` variable. After experimenting with a list-based approach that relied on a lookup in the header list to identify the field-names, a dictionary-based approach was found to be more robust and suitable. Despite being slightly more complex to implement in the rowizer, the dictionary reduces the complexity when working with the object in the later file-type stages. Mainly because for every item, the respective header field is directly available right in the same element. Especially in cases where header fields are in a different order then expected, it is easier to ask the dictionary for the key instead of searching through the whole header and data list.

```

1 class DataContainer:
2     rawData: Iterator[dict[str, str]]
3     head: list[str]
4     error: list[dict[str, str]]

```

FIGURE 5.3: Attributes of the internal data-object

Furthermore, the class has separate fields for the header and the error lines. Despite the fact that the header field technically contains duplicate information already encoded in the key-list of every dictionary-encoded item of the `rawData`-generator itself, it has

proven to be practical in many circumstances to have the header directly at disposal. Notably, the error lines are saved in memory and do not use a generator approach. This is mainly because it would significantly complicate the process of counting the amount of error lines needed for the log-entry of each file to the log table in the database. Additionally, it is to expected that the amount of error-lines is magnitudes smaller then the overall lines of a file. The potential processing of error-lines in later file-level pipeline stages like the manipulator or a second analyzer-iteration could also lead to complex problems with the generators, since they are chained throughout the pipeline and only consumed in the import-stage at the end, in order not to buffer the file in memory. Every file-level pipeline stage creates a completely new `dataContainer` object based on the one that was received in combination with changes made by it. This approach allows to chain the generators in a way that one final data-line iteration in the import-stage issues multiple iterations e.g. in the analyzing-stage (e.g. because a faulty line had to be skipped).

5.1.2 INSTRUCT

In addition to the data itself, many pipeline stages need meta information about the specific file that is currently traversing through the pipeline. Since the framework is designed to process multiple import-files in one execution, a structure that provides the import-information to the CLI in a reproducible, automatically generatable way is needed. This task is fulfilled by the Instruct-JSON illustrated in Figure 5.5. One JSON element represents one file that should be imported with all necessary meta information like the path to the file, the import-type that this file should be handled with, the table in which it should be imported and the optional date field that gets added to every data line. It also can contain a force flag (as illustrated in the second element of Figure 5.5). If this is set, the analyzer will ignore a potential double import and continue with importing anyway. If it does not occur in the JSON, it is implicitly set to false. Notably, the framework is also able to handle this data in YAML format instead of JSON.

```

1 class Instruct(BaseModel):
2     filepath: str
3     import_type: str
4     table: str
5     date: Optional[str] # default: day of import
6     force: bool = False
7     bad: bool = False
8     to_close: list

```

FIGURE 5.4: Implementation of the instruction object

```

1  [
2      {
3          "filepath": "/path/to/file1.csv",
4          "type": "foo",
5          "table": "foo_table",
6          "date": "2025-11-20"
7      },
8      {
9          "filepath": "/path/to/file2.csv",
10         "type": "bar",
11         "table": "bar_table",
12         "date": "2025-11-21"
13         "force": true
14     }
15 ]

```

FIGURE 5.5: Syntax of the Instruction JSON file

In order for the framework to be able to work with this informations internally, the individual elements get converted into a list of **Instruct** data-class elements with the syntax illustrated in Figure 5.4. The objects in the list are then issued to individual pipeline executions by the control program. The **Instruct** class mirrors the same information as the JSON element and complements it with a **bad** flag, which can be set by pipeline stages if they asses that this file cannot be imported. If the flag is set, all later stages check this when the executor calls them and directly abort their task, handing the control flow to the next stage without action. A flag is needed because individual pipeline stages do not posses the power to divert the control flow directly to the next file. Once a pipeline is initiated, the executor executes all stages before returning to the control program.

The last element of the **Instruct** class is the **to_close** list, which contains all file descriptors used throughout the execution. Because of the generator approach, it isn't trivially possible to use **python** context manager blocks for file accessing, since the file is not read directly in a class but handled across multiple objects throughout the whole pipeline. In order not to pollute memory and potentially damage the files, a reference to every pointer is stored in this list. After the successful import of all data lines, the import-stage lastly executes the **close()** operation on all elements in that list before ending the pipeline for this file.

The **Instruct** class is used throughout all pipeline stages and represents a file in the framework.

5.1.3 CH-ACCESS

The communication between the framework and the DBMS is orchestrated by the official `clickhouse-connect` [10] library directly provided by the makers of *ClickHouse*. This ensures that the import and communication between the framework and the database is as efficient as possible while also providing APIs for all necessary database features. *ClickHouse* is a network based database management system. To communicate with it, `username` and `password` is required which the system expects as mandatory command-line arguments. The URL of the DBMS as well as the specific database is currently hard-coded to default parameters in the import-stage but could easily be converted into optional arguments for the command-line. Accordingly, the framework expects that a instance is running on the same machine. Because of this, it is currently not supported to import into different databases inside a DBMS instance or to other instances with URLs different to `localhost`. After being extracted from the command line, the information is stored in a proprietary data-class shown in Figure 5.6. This object is created once per framework execution and propagated through the executor to the import stage that connects to the server using this information.

```

1 @dataclass
2 class ChAccess:
3     host: str = "localhost"
4     username: str = "default"
5     password: str = "default"

```

FIGURE 5.6: Implementation of the clickhouse-access class

5.2 MODULARITY

As addressed in Section 4.2, the design relies heavily on modularity not only in regard to the individual pipeline-stages, but also regarding the pipelines execution itself. This means that as illustrated in Figure 5.1, every stage-class is abstract and has multiple implementations in different subclasses. The only exception to this is the importing stage (in Figure 5.1 described as `ClickHouse` class), where a single universal implementation suffices for reasons already discussed in Chapter 4. Every pipeline stage has its own unique identifier that indicates which file-, import- or compression-type it can process. Those, listed in Table 5.1, are usually `String` variables declared in the superclass and statically overwritten during subclass-implementation.

In order for `python` to be able to auto detect which subclasses with which functionalities are available at execution-time, the framework relies on import-time side effects. The

TABLE 5.1: Overview of subclass identifiers

Stage	Variable	Example
Analyzer	<code>import_type</code>	"blacklist"
Rowizer	<code>logic_type</code>	".csv"
Decompressor	<code>file_ending</code>	".zst"

directory-structure of the framework illustrated in Figure 5.7 is organized in a way that all subclasses of a pipeline-stage are grouped in separate directories (i.e. analyzer implementations in directory **Analyzer**, rowizers in directory **Rowize**, etc.). This process is identical to all modular pipeline stages, however in the following exemplary described for the Analyzer:

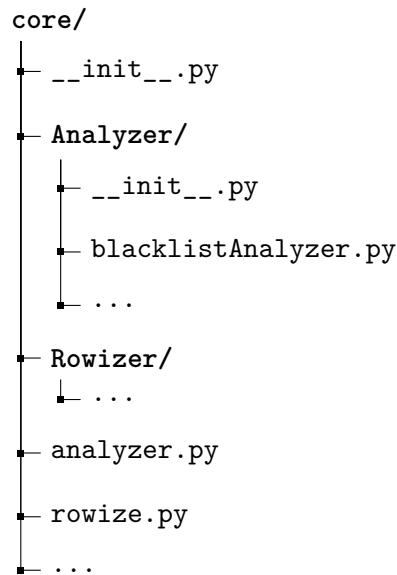


FIGURE 5.7: Excerpt of the file structure

When the framework gets called, the code in the `__init__.py` file gets executed per convention, because the **Analyzer** directory is imported during the initial control flow. The code in that file iterates and dynamically imports all the `.py` files in its directory, each containing one analyzer-subclass. This triggers the `__init_subclass__` function of each class, that registers the import-type (cf. Table 5.1) of the class in the Analyzer-Registry. Once the control flow reached the point where the executor needs to choose a specific analyzer subclass for the first import-element, it consults the Registry which returns the class-name of the suitable analyzer using `python classmethods`, if existent. The executor then instantiates the class returned from the registry and uses it in the pipeline. Said process is repeated for every pipeline instance (i.e. for every file that gets

imported).

This plugin-pattern allows users to implement new import-types simply by creating a new file in the **Analyzer** directory and coding a new subclass without any changes to the core structure necessary at all. It also allows for easy replacement of current functionality (e.g. if a secondary **blocklist** import method is needed).

Implementing new functionality for other stages follows the same principle.

5.3 CONTROL PROGRAM

After taking a closer look at the implementation of the pipeline illustrated in Figure 5.1, this section focuses on the control program that manages the pipeline executions.

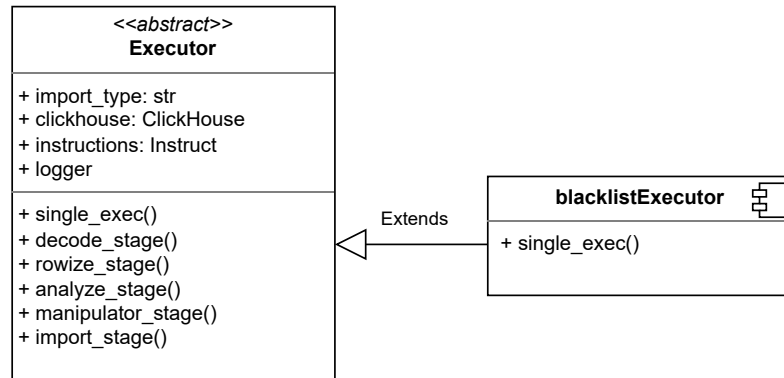


FIGURE 5.8: UML Diagram of the Execution classes

As mentioned in Section 4.2 and illustrated in Figure 4.1, the task of the **Executor** class, which is displayed in Figure 5.8, is to execute the pipeline specifically tailored to the respective import-type. Similar to the the pipeline stages, the **Executor**-class is abstract and every import-type implements its own modular subclass. The main-class contains universal wrapper functions for each pipeline stage that handles the instantiation of the respective stage that work with the same registry-constructs as the pipeline stages described in section 5.2.

All the necessary information gets pulled out of the **instructions** variable, containing the **Instruct**-Object of the file the control program assigned to this **Executor** instance. Notably, the **import_type** variable of the class is not a copy of the same field in the **Instruct** variable, but a field that is defined only in the subclasses to identify the individual executor implementations for said registries.

Accordingly, for the framework to support a new import-type, both an **Executor** and an **Analyzer** implementation is needed. The individual implementations only need to

define the import-type and arrange the wrapper functions of their superclass in the overwritten `single_exec()` function.

The **Executor** itself gets called from the outer control program. This part of the framework is not object-oriented and consists of a few static functions. It is in charge of extracting the CLI arguments, reading the in Section 5.1.2 mentioned Instruct-JSON object and funneling its contents into the individual **Instruct** elements.

When the framework is started, it begins in the `main` function where it proceeds to read the CLI arguments through the `python argparse` [11] library. It then reads the Instruct-JSON file and converts it into the list of Instruct objects. If the CLI flag *force* was set, it sets the force-flags in all Instruct-objects to true, overwriting the individual flags set in the JSON elements. After that, the function proceeds to create the **ChAccess**-object (cf. Section 5.1.3) from the given CLI parameters. Finally, dependent on the parallelism-level that defaults to 1 if no flag was given in the CLI, it creates the individual **Executor**-objects and assigns each an **Instruct-Element** from the list. This is either done sequentially or in parallel using the `python pool` functionality [12].

Notably, every object that is involved in the pipeline, from the individual stages to the executor is never reused and always freshly created for every file that gets imported. Meaning the whole pipeline is instantiated freshly for every file and thus normally multiple times per framework execution, depending on the number of files to import in the Instruct-JSON. This also means that the connection to the DBMS gets renewed for every file, since the import-stage is also not reused. Accordingly, the batching of the import-lines mentioned in Section 4.3.5 is limited to file-borders. Even if the batch-size definable via a CLI flag is greater then the amount of remaining lines or even total lines of a file, it is not possible to fill up the batch with lines from the next file as this would vastly increase complexity. The fact that the next file might be destined for a different table or even be a completely different import-type would further complicate the process.

5.4 LOGGING AND USER INTERFACE

As declared in Section 3.3, logging is a key feature of *Importheus*. The functionality can generally be split into three categories, each serving a different purpose:

- CLI-log
- log-file
- log-table

The log-table is used primarily to track which files were imported and how many error lines they contained. As explained at the beginning of this Chapter, the analyzing stage is the only instance of the framework that works with this table to check for double imports and to document the import-history in a for researchers evaluable way. The log-table does not contain any events, problems or errors the framework might have faced.

TABLE 5.2: Overview of log-levels from [13]

Level	Meaning	Example
debug	messages that show the control flow of the framework in detail	singular data-line analysis
info	status messages that signal the control flow of the framework	entering a new stage
warning	indication that something remarkable happened	double import detected
error	indication of a recoverable problem that did not lead to an early program termination	a faulty import-file-path
critical	indication of a major problem leading to premature termination	Instruct-JSON not readable

On the other hand, the primary goal of the log-file as illustrated in Figure 5.9 is to keep track of the general system status, allowing the user to retrace what the tool did. This might be especially useful for debugging purposes, e.g. to detect why a line was found to be invalid or what happened before an unexpected crash. Since the program is intended to handle multiple imports in one execution, the pipeline was implemented with a special focus on robustness in order to prevent as many unexpected crashes that would jeopardize the import of proceeding unproblematic files as possible. If one of the exception-handling blocks catches an exception, a meaningful error-message is written to the log file that explains what went wrong without stopping the execution. While the import of the file that caused the exception might fail, the goal is that problems in one pipeline execution don't affect the import of other files. In the worst case, the execution continues with the next file. To further clarify what the framework is doing, there are also info-logs throughout the whole system, e.g. when starting or finishing the import of a file or when entering the next pipeline stage. Generally, the log-messages in the log-file make use of the log-levels provided by the library. Table 5.2 explains the individual levels.

All levels above **info** are additionally written directly to the terminal, providing live

interaction with the user. If the verbose flag in the CLI is set, info-messages also get printed to the terminal.

Another concept that fosters readability of the log-file are the module-level logger concept [13]. As illustrated in Figure 5.1 and Figure 5.8, every class contains its own logger that is named after the stage it is in. This name is printed at the beginning of each line together with a timestamp as shown in Figure 5.9, automatically explaining from which part of the program the log-entry stems from.

```

1 15:28:40 [INFO] importheus: Importheus by F. Wimbauer
2 15:28:44 [INFO] importheus: Read Instruction file...
3 15:28:46 [INFO] importheus: Connect to Database...
4 15:28:50 [INFO] importheus: Setup SUCCESS. Starting file processing...
5 15:28:53 [INFO] importheus: Start sequential import of file(s)...
6 15:28:57 [INFO] importheus.execution: Decoding...
7 15:28:58 [INFO] importheus.execution: Analyzing...
8 15:29:05 [CRITICAL] importheus.analyzer:
9 Potential Double Import of file /blacklists/2025/11/2025-11-01/abuse_ch_feodo.ip.zst.
10 Skipping this file
11 ...

```

FIGURE 5.9: Exemplary excerpt from the log-file

Additionally to the logs, a import-status bar is used in the CLI to display the import progress on a file-level. For this, the `tqdm` library [14] is used. Since the verbose logging output would interfere with the `tqdm` progress bar, the logging-output gets written to the terminal through the `tqdm.write()` functionality.

CHAPTER 6

RESULTS

This Chapter takes a closer look on the results that were achieved with the implementation of *Importheus*, its impact for the current research workflow as well as challenges that were faced during development.

The tool is already an established part of the productive scientific process at the chair. Advisors argued that it sped up the procedure of importing by around 5 to 10 times from collecting the files until the queryable database. It currently supports `.zst` and `.zip` compressed files in the decompression stage. The rowizing-stage is able to work with `.csv` as well as `.json` encoded files and the analyzer-stage currently has implementations for the import-types `blacklists` and `apnic-aspop`. Pipelines with arbitrary combinations of the listed implementations work as intended. In accordance to that, it replaced the manual import process of these types fully while implementations for further types are already under development. The tool was tested on the chairs server hardware where it will be deployed in the future and proven to be reliable with up to 10 files per execution, each with over 2000 lines of data. However, under the current circumstances, the tool is only limited by the hardware it is running on. That aside, it is potentially able to import dozens of files per run.

Looking back to the requirements listed in Table 3.1, we can see that most of the requirements are met by the current implementation of *Importheus*:

- **R1 and R2: Easy Expandability for new Formats**

Thanks to the modular design of the decompression and rowizing stage that is described in Section 5.2, it is possible to implement new file formats as well as decompression formats without the need to change or understand the core pro-

gram. This also applies for the implementation of new import-types that follow the same principle.

- **R3: Import Speed**

While the actual speed of the import is not the biggest concern, the framework performs reasonably fast in a way that is still faster than handling the import process manually.

- **R4: Memory Usage**

Importheus uses a generator approach throughout the whole file processing pipeline, never storing more than one final data line in memory throughout analyzing and buffering only enough lines before import to make use of batch imports into the database. Since the batch-size is definable by the user, the amount of needed memory is transparent and thus manageable for the user.

- **R5: CPU Usage**

In order to maximize CPU usage, the framework can import multiple files in parallel at a user defined level of parallelism. This not only increases throughput but also allows to tinker the execution to the properties of the hardware on which it is running.

- **R6: Database Size**

To optimize the size of the database, the tool optionally offers full support for `clickhouse codecs` [9], which are internal compression optimizations for *ClickHouse*. In combination with the batch importing, it is ensured that the imported data is stored most efficiently.

- **R7 and R9: Analysis of individual Data-Lines**

The analyzing stage checks each data line for plausibility regarding the length of the line, the individual data-types and the order of the elements in a line. On a file level the stage checks if the given file contains the necessary fields to be imported in the desired table. This guarantees a correct database that contains no broken lines or commentary lines.

- **R8: Detecting Duplicates**

Beside checking each line, the analyzing stage consults the import-log table before processing a file to prevent accidentally importing a file multiple times. Additionally, the force-flag in the CLI provides full control in case where double imports are explicitly wanted. This makes the database unambiguous.

- **R10: User Interface**

The framework can be controlled via the CLI that provides useful help messages

if needed. All necessary parameters for the execution can be set by flags and the import files are given in the in Section 5.1.2 mentioned and in Figure 5.5 illustrated Instruct-JSON format that allows for the import of multiple files per framework-call. During execution, the tool reports its status through a progress bar and info messages directly in the CLI

- **R11: Automatic Import**

While *Importheus* is designed in a way that allows for automatic execution in the future, there is currently no implementation of a wrapper tool that would take this role.

6.1 DEVIATION FROM THE DESIGN

While the framework works as intended at this time, not all design ideas and implementation goals were met due to the limiting time frame of the thesis.

The manipulator stage referred to in Section 4.3.4 is currently implemented only as a dummy-class without any functionality. It exists, but passes on the internal data-object without taking any action. For this reason it isn't illustrated in Figure 5.1. Nevertheless, the foundational structure is there, allowing for easy implementation in the future. Concerning functionality, this means that it is currently not possible to correct import data in any form. This also concerns the error lines from the analyzing stage, which are discarded before import, as there is no mechanism existent that would be able to deal with them.

Furthermore, auto-detection of the file ending for the selection of the rowizing-stage-implementation like described in Section 4.3.2 is not implemented. This is due to the fact that the `blacklist`-files that were mainly used to initially test the framework in early development phases, didn't follow the naming scheme illustrated in Formula 4.1. Thus, the wrapper function for the rowizing-stage in the `Executor` class (cf. Figure 5.8) currently expects an additional argument for the file ending. Perhaps an approach using the `python mimetype` [15] functionality would make this step more robust. Also the idea to provide default header information in case a file does not contain any headers was not deemed as practical. It houses the the risk of bad import data in the frequent case where the header column is in a different order then expected. Despite the type-check, it could happen that two columns with the same type are interchanged, making the risk of bad imports not worth the benefit of importing a few more files. Accordingly, files that do not contain any header information are directly skipped using the in Chapter 5 described mechanisms.

During further testing of the framework, it was noticed that some import-types, often

```

1 ...
2 443,abc.de,1768,,0,0,"["tls:ServerKey","tls:x509"]",,,,0,"[42,10,80]",,,,
3 443,vzc.global,1768,0,0,"["tls:ServerKey","tls:x098"]",,,,,,"[10,80]",,,,
4 443,locator.ie,1768,0,0,"["remote_error"]",,,,,,"[112,40]",,
5 ...

```

FIGURE 6.1: Excerpt of file with optional fields

types that rely on JSON or YAML formats, might contain optional fields. While the current approach of rejecting all lines that are not complete is efficient for types that always have the same length (and where lines are actually broken if they don't), it leads to a high amount of error-lines for variable length import-types, loosing valuable data. The framework currently only imports those lines, that contain information for all declared fields to avoid empty items in the database. This policy leads to clean data, but also to cases where only a one-decimal percentage was successfully imported. Specifically for files with optional fields that only sometimes complement the normally shorter lines with extra data as illustrated in Figure 6.1, the import is currently not optimal. This problem could possibly be solved by making the decision of discarding or importing lines with missing items import-type dependent.

The mentioned example also contains nested JSON elements that are currently only saved as string elements without further handling, since there is no clear way to represent them in the same table.

6.2 POSSIBLE IMPROVEMENTS

When looking again at Figure 5.1 in conjunction with the informations of Section 4.3.3, it becomes obvious that the analyzer contains the most functionality of the whole system. This aspect grew naturally during the development of the framework, as it became clear that the tool had to carry additional functionality like the database table-creation mechanism that is first and only used in the analyzer. Thus the straight forward decision was met to implement it directly in this stage. While this approach works, the fact that one pipeline stage needs an internal reference to another one is not ideal from an architectural perspective. In addition to that, it would probably be adding to code maintainability and readability to split the analyzing stage with its current functionality in multiple smaller stages like listed in Table 6.1.

Furthermore, the whole type checking currently implemented around the `typeDict` construct, as explained at the beginning of Chapter 5, is rather inconvenient for more complex data-types. Real world use showed that relying on the user to define a correct lambda-function and a correct `python` type in order to check the data might lead to

TABLE 6.1: Overview of potential analyzer sub-stages

Stage	Description
Table Creation	stage that checks if table for import-type exists, creates it if not
Header Control	checks if the header is identical to the table header of the import-type
Line Control	analyzes each line
Logging	writes entry to the log-table

overseen edge cases and thus false positives and false negatives during decision-making. Continuing that point, the checking can get quite complex for more difficult types like the in Figure 6.1 described nested `JSON`-elements. Especially those types that require not only semantic but also syntactic analysis are hard to cover with the current approach. While implementing a solution with `pydantic` would be an option, the most promising approach would be the internal type checking functionality of `clickhouse-connect` [10]. It is able to directly translate the *ClickHouse* types into `python` logic.

While the current registry approach described in 5.2 works as intended, this part of the framework would strongly benefit from more explicit module management. This would also allow the tool to be installable via `pip`, making the execution even easier while further increasing portability and expandability.

CHAPTER 7

CONCLUSION

This Chapter summarizes the thesis and gives an outlook on possible future work that could further improve the framework.

7.1 SUMMARY

With the implementation of *Importhesus*, the process of importing Internet measurement data was significantly improved.

Researchers no longer need to fall back to various scattered scripts and proprietary importer from various authors, but have the option to use a modern, modular, well documented, consistent and dynamically expandable framework that is adaptable to their needs. The modular approach guarantees that the framework can be expanded for new import-types and file formats without the risk of destroying existing workflows that already rely on it. Furthermore, thanks to features like broken line detection, double import prevention and type validation, the tool enables higher quality datasets, further streamlining the analysis during research that relies on that data. Since the framework lays special focus on efficiently configurable CPU usage, as well as minimal memory and storage requirements, it can be used both on high-end servers, where it processes millions of lines, but is also usable for a small, experimental imports of a few files on the local machine of a researcher.

7.2 FUTURE WORK

Apart from the in Section 6.2 mentioned improvements to the current situation, there are a few general concepts that could contemplate *Importhus* in the future, to provide even more flexibility, code readability and functionality.

The most obvious thing where future work could improve the framework is added support for more import-types, compression-formats and data-formats in the form of more decompressor, rowizer and analyzer implementations according to Figure 5.1. This could also extend to more complex import-types like illustrated in Figure 6.1 that contain optional fields or JSON-structures inside of a single line-element.

Especially for such cases, where the processing complexity of a singular data-line might surpass the normal type check and header coherence control tasks, extended parallelism not only on a file-pipeline level but nested on a line level would be an interesting topic for future improvements of the framework.

For these use-cases, it might also be interesting to take a closer look on the optimization potential of the internal data management explained in Section 5.1, as there might be faster options than `python dataclasses`. For the before mentioned processing of nested lines, the current `dataContainer` from Section 5.1.1 might not be sufficient.

In order for the framework to be executed automatically on a server, some kind of wrapper program is needed that is able to call the framework without human assistance. This wrapper needs to be able to construct the Instruct-JSON illustrated in Figure 5.5 or directly provide `Instruct-Objects` (cf. Figure 5.4) for the control program by itself. In accordance to that, it needs to have configurable parameters that provide a basic overview when and where new data comes in, in order for it to schedule the import-runs. Furthermore, it needs to know the import-types of all the files through some sort of lookup-logic.

It would be conceivable that this wrapper might even be part of the framework itself, letting the user choose directly in the CLI, if the program should be executed only once with a manually given Instruct-JSON as it is done at the moment, or if the framework should be executed in auto-mode, with an import pattern that will be executed in a loop after a defined time frame. The time frame could be given as a CLI flag or be defined together with the import pattern in a similarly modular approach, just like the different patterns for the diverse import-types (cf. Figure 5.1 and Figure 5.8).

In a use case where the tool orchestrates regularly repeated import-runs on its own, it would make sense to have a more sophisticated user interface in order to easily monitor the system state. Features like a REST-API implemented with e.g. *FastAPI* [16] in conjunction with a small web-interface that illustrates basic metrics of the running

importer, come to mind to provide that. Such a web interface might also allow for small adjustments or interactive control in the future.

Furthermore, it was observed that *ClickHouse* has an upper bound limit for ingestion of new lines. If this threshold is exceeded, problems occur. To control the ingestion-flow of the import-stage it would make sense for the user to be able to declare not only a batch-size, which is already possible, but also a max ingestion-rate value in order not to exceed the capacity of the database. This further fosters the point made in Section 3.1.3 that the actual import-speed is not a big concern.

As already mentioned in the beginning of Section 6.1, the manipulator-stage is currently without function. Its implementation in the future would allow the potential correction of different data formats and possibly even the recovery of some error-lines.

CHAPTER A

APPENDIX

A.1 LIST OF ACRONYMS

CLI Command Line Interface, UI approach of the framework

GINO Global Internet Observatory [17]

OLAP Online Analytical Processing, database category of e.g. ClickHouse

DBMS Database Management System, e.g. *ClickHouse*

BIBLIOGRAPHY

- [1] *Measurement Statistics*, Last accessed on 28th January 2026, Asia-Pacific Network Informaiton Centre. [Online]. Available: <https://www.apnic.net/statistics/>.
- [2] *IPv6 Blocklists*, Last accessed on 28th January 2026, abuse.ch. [Online]. Available: <https://feodotracker.abuse.ch/blocklist/>.
- [3] *Peeringdb*, Last accessed on 28th January 2026. [Online]. Available: <https://www.peeringdb.com>.
- [4] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov, “Clickhouse - lightning fast analytics for everyone”, Association for Computing Machinery. [Online]. Available: <https://www.vldb.org/pvldb/vol117/p3731-schulze.pdf>.
- [5] *Why is clickhouse so fast?*, Last accessed on 12th February 2026, ClickHouse Documentation. [Online]. Available: <https://clickhouse.com/docs/concepts/why-clickhouse-is-so-fast>.
- [6] S. Deusch, “Analyzing the effect of domain parking on dns based research”, Online verfügbar unter: <https://example.com/bachelorarbeit.pdf>, Bachelor’s Thesis, Technical University of Munich, 2022.
- [7] R. Fontugne, M. Tashiro, R. Sommesse, M. Jonker, Z. S. Bischof, and E. Aben, “The wisdom of the measurement crowd: Building the internet yellow pages a knowledge graph for the internet”, in *Proceedings of the 2024 ACM on Internet Measurement Conference*, ser. IMC ’24, Madrid, Spain: Association for Computing Machinery, 2024, 183–198, ISBN: 9798400705922. DOI: 10.1145/3646547.3688444. [Online]. Available: <https://doi.org/10.1145/3646547.3688444>.
- [8] J. Webber, “A programmatic introduction to neo4j”, in *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity*, ser. SPLASH ’12, Tucson, Arizona, USA: Association for Computing Machinery, 2012, 217–218, ISBN: 9781450315630. DOI: 10.1145/2384716.2384777. [Online]. Available: <https://doi.org/10.1145/2384716.2384777>.

- [9] *Clickhouse compression documentation*, Last accessed on 05th February 2026. [Online]. Available: <https://clickhouse.com/docs/data-compression/compression-in-clickhouse>.
- [10] *Clickhouse-connect python library documentation*, Last accessed on 30th January 2026. [Online]. Available: <https://clickhouse.com/docs/integrations/python>.
- [11] *Python argparse library documentation*, Last accessed on 05th February 2026. [Online]. Available: <https://docs.python.org/3/library/argparse.html>.
- [12] *Python multiprocessing library documentation*, Last accessed on 05th February 2026. [Online]. Available: <https://docs.python.org/3/library/multiprocessing.html>.
- [13] *Python logging library documentation*, Last accessed on 05th February 2026. [Online]. Available: <https://docs.python.org/3/library/logging.html>.
- [14] *Tqdm documentation*, Last accessed on 05th February 2026. [Online]. Available: <https://tqdm.github.io>.
- [15] *Python mimetype documentation*, Last accessed on 11th February 2026. [Online]. Available: <https://docs.python.org/3/library/mimetypes.html#module-mimetypes>.
- [16] *Fastapi documentation*, Last accessed on 12th February 2026. [Online]. Available: <https://fastapi.tiangolo.com>.
- [17] *Global INternet Obersvatory*, Last accessed on 28th January 2026, Chair of Network Architectures and Services. [Online]. Available: <https://www.net.in.tum.de/projects/gino/>.